

Reproduction and Extension of a Stacking-Based Ensemble for Early Heart Attack Prediction

SIT720 (Machine Learning): High Distinction Task 11.1HD

Student: Ananya Gupta Student ID: 225274846

Selected paper: M. Bhagat, A. Sharma, and P. Agarwal, “An efficient stacking-based ensemble technique for early heart attack prediction” [1].

Abstract

In this report, I reproduce and critically evaluate the machine learning study by Bhagat et al. [1] on early heart attack prediction, and then develop an alternative leakage-resistant solution. I reproduced six different models: Logistic Regression, Naive Bayes, K-Nearest Neighbours, Decision Tree, Random Forest and XGBoost, together with a stacking ensemble. All classifiers were implemented together using the same 13 predictors and metrics (Accuracy, Precision, Recall, F1 Score and AUC), as reported in the original paper. The paper lacks detailed description of implementation. I therefore document all necessary assumptions for a complete reproduction and justify each of them. My reproduced stacking model was able to reach perfect 1.0 (accuracy, precision, recall, f1 score, AUC) whereas the original publication reported stacking accuracy of 0.9853 and AUC of 0.9880 for their stacking model. A more thorough exploration revealed the existence of 723 exact duplicated rows in the original dataset and 202 of the 205 records in my Part 1 test set were present with the same predictors values in the training part. For part 2, I then chose to remove rows with exact matches, made a stratified non-overlapping holdout, did preprocessing inside the cross-validation pipeline, determined Logistic Regression and Random Forest hyperparameters from training data alone, and then eventually combined the two classifiers using soft voting. The final model got a mean five-fold cross-validated AUC of 0.9254 and a held-out test AUC of 0.8777. With the same leakage-resistant protocol, a direct model comparison showed soft voting getting an AUC of 0.9254 while paper-style stacking got 0.9103. These results suggest that the main contribution of my extension is a more rigorous evaluation protocol, together with a modest ensemble improvement, rather than a higher raw accuracy.

1. Introduction and Research Context

1.1. Research Problem and Motivation

Heart disease is a major health problem, and early identification of patients at higher risk can support earlier clinical assessment and intervention. Bhagat et al. [1] investigate whether supervised machine learning can improve early heart attack prediction using routinely available clinical attributes. Their motivation is that conventional cardiovascular risk scoring approaches can be limited, while machine learning can model more complex relationships across multiple patient characteristics [1].

The objective of the selected study is to build an accurate heart disease prediction framework by evaluating several commonly used classifiers and then combining them through stacking. The paper evaluates Logistic Regression, Decision Tree, Random Forest, XGBoost, Naive Bayes and K-Nearest Neighbours before proposing a stacking-based ensemble [1].

1.2. State-of-the-Art Approaches and Research Gap

The paper reviews earlier heart disease prediction studies using individual and hybrid machine learning approaches. Examples discussed by the authors include a hybrid Random Forest and linear-model approach reporting 88.47% accuracy and feature-reduction approaches based on chi-square selection and PCA reporting very high accuracies on several heart disease datasets [1]. Random Forest and Support Vector Machine methods also appear frequently in the prior literature summarised by the authors [1].

From this context, I interpret the paper's research gap as the need for a more reliable and comprehensive prediction framework that compares several classifiers within one study and combines their strengths. Bhagat et al. address this by using a combined heart disease dataset, evaluating six classifiers, and introducing a stacking ensemble [1]. My reproduction first tests whether the reported findings can be recreated and then investigates whether the evaluation protocol itself has limitations that affect the apparent performance.

2. Dataset and Feature Set

The dataset used in my reproduction contains 1,025 observations and 14 columns: 13 predictor variables and one binary target variable. The target distribution is close to balanced, with 526 positive and 499 negative observations. No missing values were present in the dataset loaded for my experiment. The same 13 predictors used in the selected study were retained [1].

Feature	Role / type
age	Continuous
sex	Categorical / binary
cp	Categorical - chest pain type
trestbps	Continuous - resting blood pressure
chol	Continuous - serum cholesterol
fbs	Categorical / binary - fasting blood sugar
restecg	Categorical - resting ECG
thalach	Continuous - maximum heart rate
exang	Categorical / binary - exercise-induced angina
oldpeak	Continuous - ST depression
slope	Categorical - ST segment slope
ca	Categorical - number of major vessels
thal	Categorical - thalassemia-related code
target	Binary outcome

Table 1. Feature set used in the reproduction.

During initial exploration I also found 723 exact duplicate rows. I retained all 1,025 observations for Part 1 because my aim was first to reproduce the experiment using the dataset in the form associated with the selected study. I investigate the effect of duplicates separately and use that finding to motivate Part 2.

3. Part 1: Reproduction Methodology

3.1. Train/Test Split Assumption

The paper does not clearly specify every detail required to recreate the final train/test split [1]. I therefore tested plausible test-set sizes against the published accuracy values. A 20% test split gives 205 observations, and several published accuracies map almost exactly to integer numbers of correct predictions on a test set of this size. For example, an accuracy of 0.8439 corresponds to approximately 173/205 correct predictions and 0.9073 corresponds to approximately 186/205. I therefore used an 80:20 stratified train/test split with `random_state=42` as a reasonable and reproducible assumption. This produced 820 training observations and 205 test observations.

3.2. Preprocessing

The paper describes preprocessing that includes handling missing values, scaling numerical variables and encoding categorical variables [1]. However, the exact scaling implementation is not clearly specified. In my downloaded dataset, no missing values were present and the categorical variables were already numerically encoded. I therefore used `StandardScaler` as a reproducible scaling assumption for the continuous variables age,

trestbps, chol, thalach and oldpeak. The scaler was fitted on the training data only, while the categorical codes were left unchanged.

3.3. Classifiers and Stacking

I reproduced all six individual classifiers used in the paper: Logistic Regression, Naive Bayes, KNN, Decision Tree, Random Forest and XGBoost [1]. Where complete hyperparameters were not available, I used standard library settings and fixed `random_state=42` where applicable. Logistic Regression used `max_iter=1000`. I then constructed a stacking ensemble containing the same six base classifiers. The paper describes five-fold stacking but does not clearly specify the final meta-classifier [1], so I used Logistic Regression as the meta-classifier as a reasonable assumption.

3.4. Evaluation Metrics

I used Accuracy, Precision, Recall, F1 Score and AUC, matching the metrics reported by Bhagat et al. [1]. AUC was calculated using positive-class probabilities rather than hard class predictions.

3.5. Reproduction Assumptions

Implementation detail	Information available from paper	My reproduction decision
Feature set	13 predictors reported	Same 13 predictors
Train/test proportion	Not clearly specified	80:20 stratified train/test split
Random seed	Not clearly specified	<code>random_state=42</code>
Numerical preprocessing	Numerical scaling/normalisation described; exact method unclear	StandardScaler fitted on training data
Categorical encoding	Encoding described	Existing numeric categorical codes retained
Classifier parameters	Not completely reported	Library defaults where unspecified
Stacking validation	Five-fold approach described	<code>cv=5</code>
Meta-classifier	Not clearly specified	Logistic Regression

Table 2. Assumptions required to reproduce implementation details that were unclear or not fully specified in the selected paper.

4. Part 1: Reproduced Results

Model	Pub. Acc.	Rep. Acc.	Pub. Prec.	Rep. Prec.	Pub. Rec.	Rep. Rec.	Pub. F1	Rep. F1	Pub. AUC	Rep. AUC
Logistic Regression	.8439	.8146	.8196	.7638	.9090	.9238	.8620	.8362	.9204	.9297
Naive Bayes	.8439	.8293	.8250	.8070	.9000	.8762	.8608	.8402	.9185	.9043
KNN	.8585	.8537	.8648	.8571	.8727	.8571	.8687	.8571	.9304	.9540
Decision Tree	.9268	.9854	.9702	1.0000	.8909	.9714	.9289	.9855	.9702	.9857
Random Forest	.9268	1.0000	.9130	1.0000	.9545	1.0000	.9333	1.0000	.9734	1.0000
XGBoost	.9073	1.0000	.9174	1.0000	.9090	1.0000	.9132	1.0000	.9830	1.0000
Stacking	.9853	1.0000	1.0000	1.0000	.9727	1.0000	.9861	1.0000	.9880	1.0000

Table 3. Published results [1] compared with my reproduced results.

The reproduction was close to the published results for some models but differed substantially for the tree-based methods. KNN was the closest in accuracy, with 0.8537 reproduced compared with 0.8585 published. Logistic Regression and Naive Bayes were slightly lower in reproduced accuracy, while Decision Tree, Random Forest and XGBoost were substantially higher. Random Forest and XGBoost both achieved 1.0000 across all five reproduced metrics. The reproduced stacking ensemble also achieved 1.0000 for Accuracy, Precision, Recall, F1 Score and AUC, compared with the published stacking accuracy of 0.9853 and AUC of 0.9880 [1].

Published-result inconsistency. I observed that the paper contains some inconsistent performance values across different sections. For example, an earlier section reports XGBoost accuracy of 93.17% and stacking

accuracy of 96.58%, whereas the abstract and final results section report different values. For consistency, I use the results presented in the paper's final classifier results table (Table 11) as the authoritative published results for my reproduction comparison.

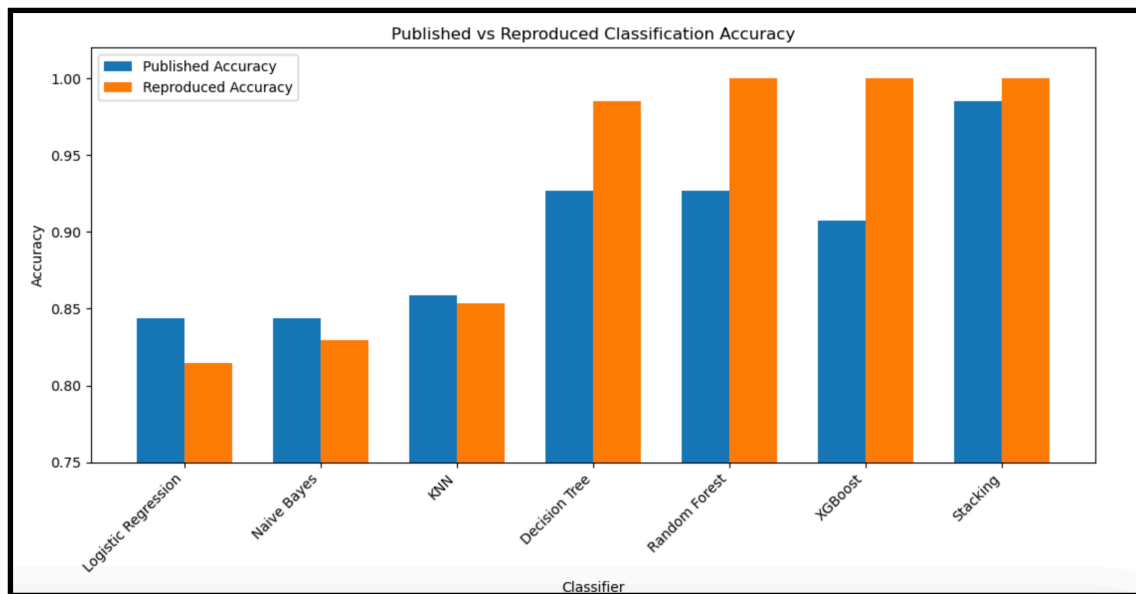


Figure 1. Published versus reproduced classification accuracy for the seven evaluated models.

4.1. Critical Analysis of Reproduction Differences

The unusually high reproduced performance led me to investigate the dataset structure rather than treating the perfect results as evidence of superior generalisation. I found 723 fully duplicated rows in the 1,025-row dataset. More importantly, 202 of the 205 observations in my Part 1 test set had an identical predictor record in the training data, corresponding to 98.54% exact predictor overlap.

This provides a plausible explanation for at least part of the unusually high performance of flexible models such as Random Forest, XGBoost and stacking. Duplicate cases divided between training and test data are a form of train-test contamination and can produce overly optimistic estimates of performance [2], [3]. However, I cannot conclude that the original paper used the same exact split or had the same 98.54% overlap because its exact random split is not available. Other differences may also result from unspecified hyperparameters, random initialisation, the assumed 80:20 split, and my choice of Logistic Regression as the stacking meta-classifier.

Therefore, I interpret Part 1 as a successful broad reproduction of the paper's modelling framework, but I treat the near-perfect reproduced scores cautiously. This finding directly motivates the revised methodology in Part 2.

5. Part 2: Motivation and Proposed Solution

My proposed solution addresses the evaluation limitation identified during reproduction. Rather than only replacing one classifier with another, I redesign the data preparation, validation and ensemble pipeline. The main changes are: (1) remove exact duplicate observations before splitting; (2) create a stratified 80:20 train/test split with no exact predictor overlap; (3) keep the held-out test set completely separate from model development; (4) perform preprocessing inside cross-validation pipelines; (5) optimise selected models using training-only five-fold stratified cross-validation; and (6) combine optimised Logistic Regression and Random Forest using soft voting.

This design is motivated by guidance that duplicate cases across train/test partitions and preprocessing outside the validation procedure can cause leakage and optimistic evaluation [2], [3]. Cross-validation is also useful when model development is based on a limited sample [4]. The objective of Part 2 is therefore not to maximise raw test accuracy at any cost, but to obtain a more defensible estimate of performance on distinct observations.

Difference from the original approach. My proposed solution is not simply a change of classifier or hyperparameters. The original reproduction uses the 1,025-row dataset and a stacking ensemble, whereas my proposed workflow first changes the evaluation design by removing exact duplicates before splitting, enforcing zero exact predictor overlap between training and testing, fitting preprocessing separately within cross-validation folds, restricting model selection and hyperparameter optimisation to the training data, and reserving the held-out test set for one final evaluation. I then replace the stacking architecture with an optimised soft-voting ensemble. Therefore, the proposed contribution combines a redesigned experimental protocol with a different ensemble strategy.

6. Part 2: Experimental Protocol

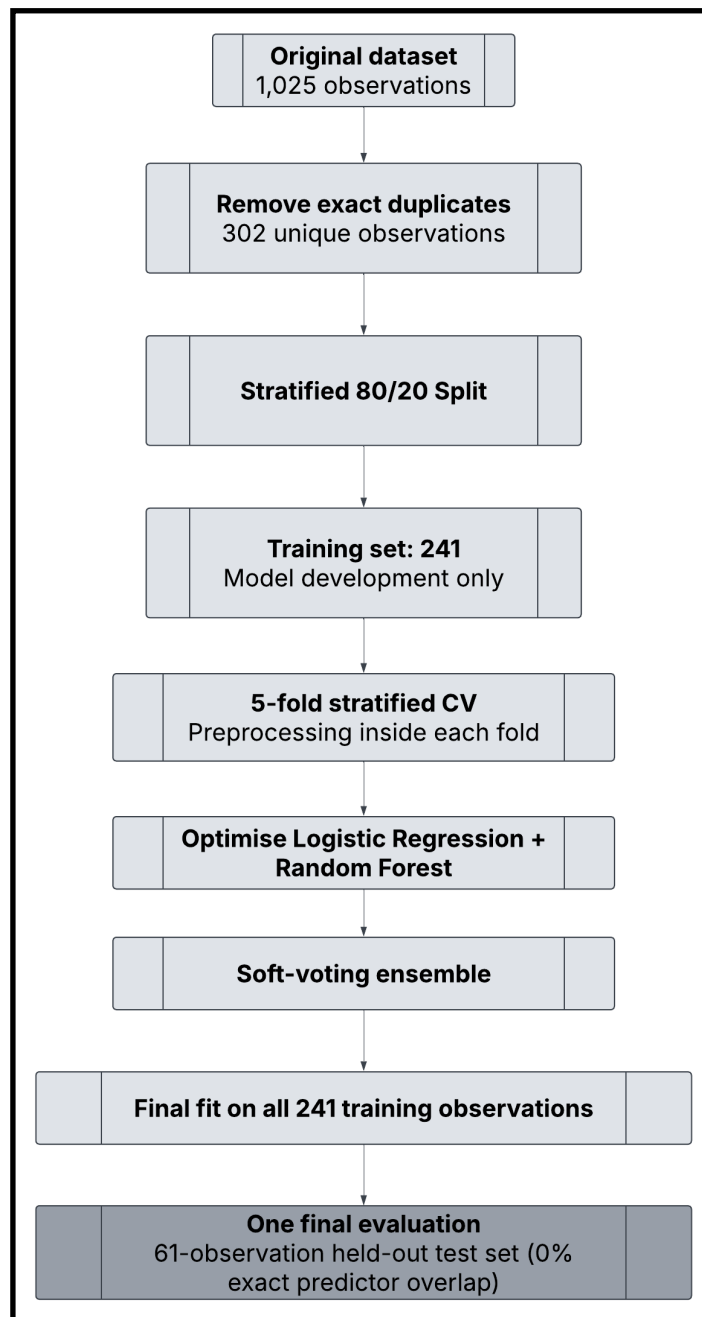


Figure 2. Proposed leakage-resistant experimental workflow. The 61-observation held-out test set is kept separate from all model development and is used only for the final evaluation (Made with LuicidChart).

6.1.Deduplication and Split

Removing exact duplicate rows reduced the dataset from 1,025 to 302 unique observations: 164 positive and 138 negative. I then used a stratified 80:20 split with random_state=42, producing 241 training observations and 61 held-out test observations. I explicitly checked the predictor rows after splitting and found zero exact test-train overlap, compared with 98.54% in Part 1.

6.2.Leakage-Resistant Baseline Evaluation

I evaluated the same six baseline classifiers using stratified five-fold cross-validation on only the 241-observation training set. StandardScaler was placed inside a Pipeline and applied only to the five continuous features, so it was fitted separately within each training fold. The held-out test set was not used for baseline evaluation, model selection or hyperparameter optimisation.

Model	Mean Acc.	Mean Prec.	Mean Rec.	Mean F1	Mean AUC	AUC SD
Logistic Regression	.8420	.8199	.9234	.8661	.9149	.0354
Random Forest	.8257	.8238	.8778	.8466	.9097	.0442
Naive Bayes	.8172	.8244	.8624	.8378	.8950	.0469
XGBoost	.7964	.8195	.8160	.8117	.8884	.0357
KNN	.8213	.8026	.9080	.8481	.8622	.0262
Decision Tree	.8007	.8064	.8393	.8193	.7969	.0202

Table 4. Leakage-resistant five-fold cross-validation baseline results on the Part 2 training set.

Logistic Regression produced the highest baseline mean AUC at 0.9149, while Random Forest produced the second-highest at 0.9097. I selected these two models for optimisation because they were the strongest AUC candidates while representing different linear and nonlinear modelling approaches.

6.3.Hyperparameter Optimisation and Soft Voting

For Logistic Regression I tuned C over 0.01, 0.1, 1, 10 and 100 and compared class_weight=None with class_weight='balanced'. The best setting was C=1 with balanced class weights, giving a mean CV AUC of 0.9156. For Random Forest I tuned n_estimators, max_depth, min_samples_split, min_samples_leaf and max_features. The best configuration used 100 trees, no maximum depth, max_features='sqrt', min_samples_leaf=4 and min_samples_split=2, giving a mean CV AUC of 0.9190.

I combined the two optimised models using soft voting, which averages their predicted class probabilities. This differs from the paper's stacking architecture because it does not train an additional meta-classifier. On the five training folds, the proposed ensemble achieved mean Accuracy 0.8461, Precision 0.8387, Recall 0.9080, F1 Score 0.8680 and AUC 0.9254, with AUC standard deviation 0.0337.

7. Part 2: Final Results

7.1.Held-Out Test Performance

Accuracy	Precision	Recall	F1 Score	AUC
0.7869	0.8125	0.7879	0.8000	0.8777

Table 5. Final performance of the proposed soft-voting ensemble on the untouched 61-observation held-out test set.

After model selection was complete, I fitted the final ensemble on the full Part 2 training set and evaluated it once on the held-out test set. The test AUC of 0.8777 was lower than the mean cross-validated AUC of 0.9254, showing that performance varied between the training-fold validation estimates and this small held-out sample.

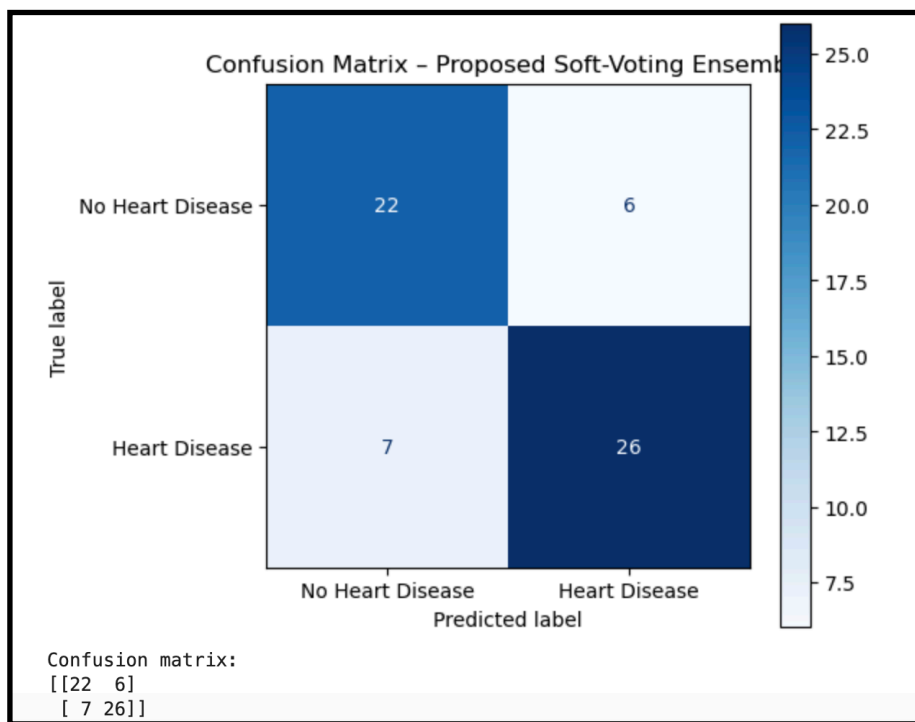


Figure 3. Confusion matrix for the proposed soft-voting ensemble on the held-out test set.

The confusion matrix contains 22 true negatives, 26 true positives, 6 false positives and 7 false negatives. The model therefore correctly classified 48 of the 61 held-out observations.

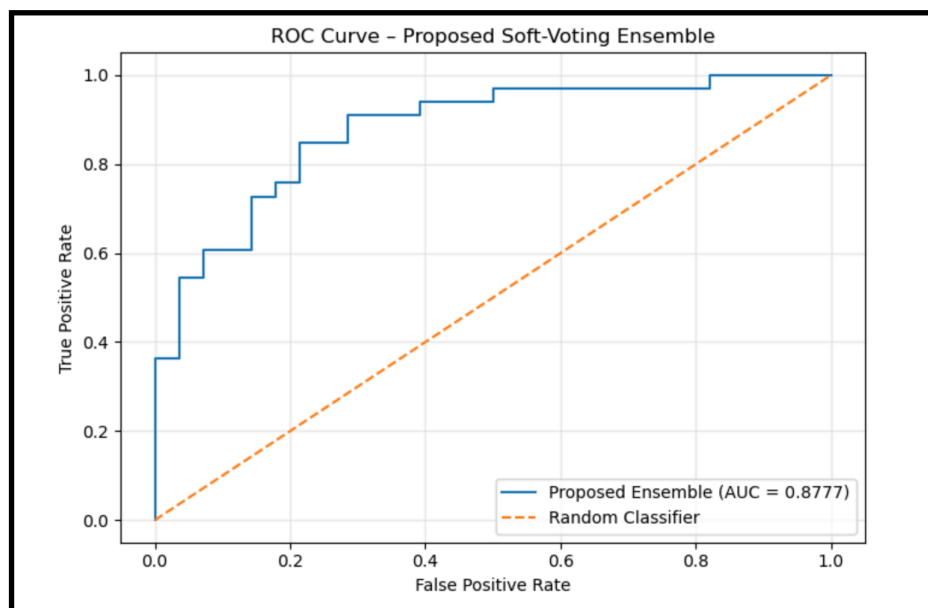


Figure 4. ROC curve for the proposed soft-voting ensemble on the held-out test set (AUC = 0.8777).

7.2. Cross-Validation Stability

Fold	Optimised LR AUC	Optimised RF AUC	Soft-Voting AUC
1	.9697	.9832	.9764
2	.8794	.9353	.9056
3	.8934	.8531	.8864
4	.9458	.9580	.9528
Fold	Optimised LR AUC	Optimised RF AUC	Soft-Voting AUC
5	.8899	.8654	.9056
Mean	.9156	.9190	.9254
SD	.0355	.0512	.0337

Table 6. Fold-level AUC comparison for the two optimised component models and the proposed ensemble.

The ensemble achieved the highest overall mean AUC and the lowest AUC standard deviation, although it was not the best model in every fold. Random Forest was highest in folds 1, 2 and 4, Logistic Regression was highest in fold 3, and soft voting was highest in fold 5. I therefore interpret the improvement as modest rather than uniform.

8. Comparative Evaluation

Approach	Accuracy	Precision	Recall	F1	AUC	Data	Exact overlap
Published stacking [1]	.9853	1.0000	.9727	.9861	.9880	1,025	Not reported
Part 1 reproduced stacking	1.0000	1.0000	1.0000	1.0000	1.0000	1,025	98.54%
Part 2 proposed soft voting	.7869	.8125	.7879	.8000	.8777	302	0.00%

Table 7. Published, reproduced and proposed final results. The paper does not report exact test-train duplicate overlap; my 205-row Part 1 test size is based on the documented 80:20 reproduction assumption.

The proposed model has lower held-out scores than both the published and Part 1 stacking results, but these values were obtained under substantially stricter conditions. The Part 1 split had 98.54% exact predictor overlap, while the Part 2 split had 0%. At the same time, deduplication reduced the available data from 1,025 observations to only 302, so the entire performance reduction cannot be attributed to overlap alone. The smaller training set and sampling variation in a 61-observation test set are also likely contributors.

8.1. Same-Protocol Comparison with Paper-Style Stacking

Model	Mean Acc.	Mean Prec.	Mean Rec.	Mean F1	Mean AUC	AUC SD
Paper-style stacking	.8297	.8263	.8855	.8516	.9103	.0387
Proposed soft voting	.8461	.8387	.9080	.8680	.9254	.0337

Table 8. Controlled five-fold cross-validation comparison using the same deduplicated training data and leakage-resistant preprocessing.

This controlled experiment separates the ensemble architecture from the effect of deduplication. Under the same Part 2 protocol, soft voting improved mean AUC by 0.0151 and mean Accuracy by 0.0164 compared with paper-style stacking. It also achieved slightly lower AUC variability. These differences are small and are based on only five folds, so I treat them as evidence of an incremental improvement rather than definitive superiority.

9. Critical Discussion

9.1. Strengths

The main strength of my proposed solution is that it responds directly to an empirical limitation discovered during reproduction. Duplicate removal reduced exact test-train predictor overlap from 98.54% to 0%. Model selection was performed only on the training set, preprocessing was fitted within each cross-validation fold,

and the final holdout was not used until model development was complete. These choices are consistent with recommendations intended to reduce train-test contamination and leakage [2]–[4].

The proposed method also provides modelling evidence beyond the protocol change. Soft voting achieved the highest mean CV AUC of 0.9254 compared with the optimised Logistic Regression (0.9156), optimised Random Forest (0.9190), and controlled paper-style stacking model (0.9103).

9.2.Limitations and Practical Implications

The most important limitation is the small effective sample after deduplication. Only 302 unique observations remained, with 241 used for training and 61 for final testing. Individual errors therefore have a noticeable effect on the reported metrics. The difference between mean CV AUC (0.9254) and held-out AUC (0.8777) also shows uncertainty in generalisation.

The Part 2 holdout is still an internal test set from the same underlying dataset, not an external validation dataset. External validation evaluates a developed model using separate data and is important for understanding whether performance transfers to other relevant populations [5]. This is especially important in cardiovascular prediction, where external-validation studies often report reduced discrimination compared with development data [6]. Future work should therefore evaluate the complete pipeline on a larger independent cardiovascular dataset.

Finally, the soft-voting improvement is modest and was not consistent in every fold. I therefore do not claim that soft voting is universally better than stacking. The stronger contribution is the combination of a leakage-resistant protocol and an ensemble that performed slightly better than paper-style stacking when both were evaluated under the same controlled conditions.

10. Conclusion

In Part 1, I reproduced the six individual classifiers and stacking ensemble from Bhagat et al. [1]. The reproduced stacking model achieved perfect scores on the selected split, slightly exceeding the published stacking result. However, I identified 723 duplicate observations and found that 202 of 205 Part 1 test observations had an identical predictor record in the training set. This made the near-perfect reproduced performance difficult to interpret as evidence of generalisation.

In Part 2, I redesigned the workflow by removing exact duplicates before splitting, creating a 0%-overlap holdout, placing preprocessing inside cross-validation pipelines, tuning models using training data only, and developing a soft-voting ensemble. The proposed ensemble achieved a mean CV AUC of 0.9254 and held-out AUC of 0.8777. Under the same leakage-resistant cross-validation protocol, soft voting also achieved a higher mean AUC than paper-style stacking (0.9254 versus 0.9103).

Overall, my results show that evaluation design can have a major effect on apparent machine learning performance. The Part 2 scores are lower than the headline results from Part 1, but they are based on distinct held-out observations and a more controlled model-development procedure. Further external validation on a larger dataset is needed before drawing conclusions about clinical generalisability.

11. Part 3: Video Presentation

Video presentation link: <https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=4d2bf71f-bbe8-48e8-9246-b4d100a50bec>

In the video I briefly explain the selected paper, demonstrate the Part 1 reproduction and duplicate-overlap finding, present the leakage-resistant Part 2 pipeline and soft-voting ensemble, show selected notebook outputs, and reflect on the limitations and future improvements. The final video is around 5 minutes, making sure I meet the time requirements.

12. Reproducibility and Source Files

Repository link: <https://github.com/ananyag270903/SIT720-Task-11.1HD>

The submitted repository contains the final Jupyter notebook, the dataset used for reproduction, and a README explaining the required packages and how to run the notebook from start to finish. The notebook has been tested using a clean kernel and Run All so that the report outputs can be reproduced.

13. Generative AI Acknowledgement

I used AI as a support tool during this task for planning the structure of my work, discussing methodological choices, and helping me review whether I addressed the assessment requirements. I critically reviewed, verified and adapted the suggestions against my own implementation, experimental outputs, the selected research paper and supporting literature. The final analysis, code execution, interpretation and submitted work remain my responsibility.

References

1. M. Bhagat, A. Sharma, and P. Agarwal, “An efficient stacking-based ensemble technique for early heart attack prediction,” *Multimedia Tools and Applications*, vol. 84, no. 30, pp. 36351–36375 (2025). DOI: <https://doi.org/10.1007/s11042-024-20064-7>
2. Stephen R Master, Tony C Badrick, Andreas Bietenbeck, Shannon Haymond, Machine Learning in Laboratory Medicine: Recommendations of the IFCC Working Group, *Clinical Chemistry*, Volume 69, Issue 7, July 2023, Pages 690–698, <https://doi.org/10.1093/clinchem/hvad055>
3. Sayash Kapoor et al. ,REFORMS: Consensus-based Recommendations for Machine-learning-based Science.Sci. Adv.10,eadk3452(2024).<https://www.science.org/doi/10.1126/sciadv.adk3452>
4. Vabalas A, Gowen E, Poliakoff E, Casson AJ (2019) Machine learning algorithm validation with a limited sample size. *PLOS ONE* 14(11): e0224365. <https://doi.org/10.1371/journal.pone.0224365>
5. Riley RD, Archer L, Snell KIE, Ensor J, Dhiman P, Martin GP, Bonnett LJ, Collins GS. Evaluation of clinical prediction models (part 2): how to undertake an external validation study. *BMJ*. 2024 Jan 15;384:e074820. doi: <https://doi.org/10.1136/bmj-2023-074820>
6. B. S. Wessler et al., “External Validations of Cardiovascular Clinical Prediction Models: A Large-Scale Review of the Literature,”*Circulation: Cardiovascular Quality and Outcomes*, vol. 14, no. 8, Art. no. e007858, Aug. 2021, doi: <https://doi.org/10.1161/CIRCOUTCOMES.121.007858>